

云计算技术课程设计报告

封面

课程名称：云计算技术

课程代码：SCAI004712

学号：2023112573

姓名：张春冉

班级：软件工程 2023-02 班

队友：2023112551 邓苏鑫

日期：2026 年春季学期

一、华为云环境信息

本实验区域选择华为云华北-北京四，Region 为 `cn-north-4`。CCE 集群名称为 `cloud-compute-cce`，集群类型为 CCE Standard，Kubernetes 版本为 `v1.34.6-r0-34.0.23`。集群包含 2 个 Worker 节点，节点系统为 Ubuntu 22.04.5 LTS，容器运行时为 `containerd 1.7.29`。镜像仓库使用华为云 SWR，组织名为 `cloud-compute-2026`。

集群通过公网 `kubeconfig` 在本地完成管理。`kubectl get nodes -o wide` 的输出显示两个 Worker 节点均为 `Ready`，版本列满足课程要求中 Kubernetes 版本不低于 1.27 的条件。图 1 预留给 `Ghostty` 终端中运行真实 `kubectl` 命令得到的截图。

【截图占位：图 1 `kubectl get nodes -o wide`。要求画面包含两个 Worker 节点、`Ready` 状态和 `VERSION` 列。】

二、第一部分：云计算平台搭建

2.1 应用容器化

本部分实现了一个两层 Web 应用。后端使用 Flask 编写，主要接口为 `/api/ping`、`/api/count` 和 `/api/httpbin`。其中 `/api/ping` 返回节点状态、学生信息和当前 UTC 时间，并尝试连接 Redis 执行 `ping_count` 自增；`/api/count` 读取 Redis 中的访问计数。`backend/requirements.txt` 中包含 `flask==3.0.0`、`redis==5.0.1` 和 `requests==2.32.3`，其中 `requests` 作为课程要求中的自选 Python 包。

前端使用 Nginx 托管静态页面，首页展示课程设计名称、学号、姓名、队友信息和系统架构。页面中的按钮会请求 `/api/ping`，由 Nginx 将 `/api/` 路径反向代理到后端节点。前端首页中包含 2023112573 张春冉，可用于答辩验收识别。

镜像构建采用多阶段 Dockerfile。后端镜像先在 builder 阶段安装 Python 依赖，再将依赖复制到运行镜像，减少运行时镜像内容。前端镜像基于 nginx:1.25-alpine，将 nginx.conf 和 static/ 目录复制到镜像中。实际推送时使用 docker buildx build --platform linux/amd64 --provenance=false --push，避免本机 Apple Silicon 构建出的镜像架构与 CCE Worker 节点不一致，也避免 SWR 对 BuildKit provenance manifest 的解析问题。

已推送到 SWR 的镜像如下：

镜像	Tag	地址
backend	v1	swr.cn-north-4.myhuaweicloud.com/cloud-compute-2026/backend:v1
frontend	v1	swr.cn-north-4.myhuaweicloud.com/cloud-compute-2026/frontend:v1

本地单元测试使用 .venv/bin/python -m unittest discover -s backend/tests 运行，结果为 Ran 1 test ... OK。测试记录保存在 report/evidence/20260612-194545-verification-complete.txt。

【截图占位：图 2 本地 docker compose up --build 运行截图，需包含 backend、frontend、redis 任务启动日志。】

【截图占位：图 3 SWR 镜像仓库截图，需包含 backend:v1 和 frontend:v1。】

2.2 CCE 集群搭建

CCE 集群已创建完成，并通过 kubectl 连接验证。节点信息如下：

NAME	STATUS	ROLES	VERSION	OS-IMAGE
192.168.0.113 LTS	Ready	<none>	v1.34.6-r0-34.0.23	Ubuntu 22.04.5
192.168.0.214 LTS	Ready	<none>	v1.34.6-r0-34.0.23	Ubuntu 22.04.5

该结果说明两个 Worker 节点均处于可调度状态，版本信息也已在输出中显示。实验过程中曾使用 CloudShell 初始化 kubectl，后续为便于自动化部署，又为 CCE API Server 绑定公网地址并下载公网访问 kubeconfig，在本地完成部署、镜像推送后的重启和验证。

【截图占位：图 4 CCE 控制台集群概览截图，需包含集群名称、运行中状态、Kubernetes 版本和 2/2 节点状态。】

2.3 应用部署

K8s 部署文件位于 k8s/ 目录。后端 Deployment 副本数为 2，Redis Deployment 副本数为 1，前端 Deployment 副本数为 1。后端容器配置了 CPU 和内存的 requests 与 limits，并通过 envFrom 引用 ConfigMap 中的 Redis 地址，通过 secretKeyRef 引用 Redis 密码。Redis 只通过 ClusterIP Service 暴露给集群内部，后端和前端使用 LoadBalancer Service 对外暴露。

部署过程中遇到两个华为云 CCE 相关问题。镜像拉取开始报 401 Unauthorized，排查后发现集群中已有 default-secret 类型为 kubernetes.io/dockerconfigjson，但默认 ServiceAccount 没有引用该 Secret，因此在后端和前端 Deployment 中增加 imagePullSecrets。随后镜像拉取错误变为 not found，说明认证已通过但 SWR 中缺少镜像；完成本地构建和推送后，Pod 正常拉起。LoadBalancer Service 开始长期停留在 <pending>，事件中提示 service annotation(kubernetes.io/elb.id) or service.spec.loadBalancerIP is not defined，因此在 Service 注解中加入 kubernetes.io/elb.autocreate，让 CCE 自动创建公网 ELB。

当前运行状态如下：

backend-97b4d5c4-drp84	1/1	Running
backend-97b4d5c4-qtjlg	1/1	Running
frontend-658dcb676-2dgdj	1/1	Running
redis-ff6998b77-nbpzb	1/1	Running

后端 LoadBalancer 公网地址为 1.92.103.90，前端 LoadBalancer 公网地址为 119.3.252.161。访问后端接口返回：

```
{"redis":"connected","status":"ok","student":"2023112573 张春冉","time":"2026-06-12T11:36:52.137486+00:00"}
```

访问前端的 /api/ping 也返回同样的 JSON 结构，说明公网入口、前端 Nginx 反向代理、后端 Flask API 和 Redis 之间的链路已经连通。最终验证截图见 report/evidence/ghostty-final-status-cropped.png，完整文本证据见 report/evidence/20260612-193650-final-success-status.txt。

【截图占位：图 5 kubectl get pods -o wide，需显示 backend 两个副本、frontend、redis 均为 Running。】

【截图占位：图 6 kubectl get svc，需显示 backend-svc 和 frontend-svc 的 LoadBalancer 公网地址。】

【截图占位：图 7 浏览器或 curl 访问 `http://1.92.103.90/api/ping`，返回 `status=ok` 且 `redis=connected`。】

【截图占位：图 8 浏览器访问 `http://119.3.252.161/`，页面显示学号、姓名和架构信息。】

2.4 持久化存储

Redis 使用 PVC `redis-data-pvc` 挂载到容器内 `/data` 目录，PVC 的 `storageClassName` 为 `csi-disk`，容量为 10Gi。kubectl get pvc 输出显示 PVC 已绑定：

NAME	STATUS	CAPACITY	ACCESS MODES	STORAGECLASS
redis-data-pvc	Bound	10Gi	RWO	csi-disk

当前报告已保留 PVC Bound 证据，截图见 Ghostty 终端截图。课程要求中的“写入 testkey、删除 Redis Pod、重建后读取 testkey”尚需单独执行并截图。建议补充命令如下：

```
REDIS_POD=$(kubectl get pod -l app=redis -o
jsonpath='{.items[0].metadata.name}')
kubectl exec "$REDIS_POD" -- redis-cli -a cloudcompute2026 SET testkey
hello
kubectl exec "$REDIS_POD" -- redis-cli -a cloudcompute2026 GET testkey
kubectl delete pod "$REDIS_POD"
kubectl get pods -w
NEW_REDIS_POD=$(kubectl get pod -l app=redis -o
jsonpath='{.items[0].metadata.name}')
kubectl exec "$NEW_REDIS_POD" -- redis-cli -a cloudcompute2026 GET
testkey
```

该组截图应包括 PVC Bound、写入结果、删除 Pod 后新 Pod Running、重新读取 hello 四类信息。执行时 Redis CLI 可能提示密码暴露在命令行中，这是课程实验环境可接受的提示，不影响验证结论。

【截图占位：图 9 kubectl get pvc，需显示 redis-data-pvc 为 Bound。】

【截图占位：图 10 Redis 写入 testkey=hello。】

【截图占位：图 11 删除 Redis Pod 并等待新 Pod Running。】

【截图占位：图 12 重建后读取 testkey，返回 hello。】

2.5 ConfigMap Volume 挂载

前端 Nginx 反向代理配置存放在 ConfigMap `frontend-nginx-conf` 中，并通过 Volume 挂载到 `/etc/nginx/conf.d/default.conf`。Pod 内实际文件内容如下：

```
location /api/ {
    proxy_pass http://backend-svc:80/api/;
```

```

    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}

```

Volume 挂载适合 Nginx 配置、证书、应用配置文件等“文件形式”的配置。配置改变后，容器内对应文件可以被更新，适合需要由应用按文件路径读取的场景。envFrom 更适合主机名、端口、环境标识等键值型配置，应用从环境变量读取即可。本实验中后端使用 envFrom 读取 Redis 地址和端口，前端使用 ConfigMap Volume 挂载 Nginx 配置，两种方式分别对应键值配置和文件配置。

本次部署过程中曾重建 backend-svc，导致 Nginx 进程仍持有旧的 Service ClusterIP。通过 `kubectl logs deploy/frontend` 可看到 Nginx 访问旧 upstream 超时。重启前端 Deployment 后，Nginx 重新解析 backend-svc，前端 `/api/ping` 恢复正常。该过程说明 ConfigMap Volume 解决的是配置文件挂载问题，下节发现和 Nginx DNS 缓存仍需要结合进程重载或 Pod 重启处理。

【截图占位：图 13 `kubectl exec` 进入前端 Pod 后 `cat /etc/nginx/conf.d/default.conf`，需显示反向代理到 `backend-svc:80`。】

【截图占位：图 14 修改 ConfigMap 后重新 `cat /etc/nginx/conf.d/default.conf`，需体现配置文件已更新。】

2.6 HPA 弹性伸缩

后端 Deployment 已配置 HPA，目标 CPU 利用率为 60%，副本范围为 1 到 4。当前 `kubectl get hpa` 输出如下：

NAME	REFERENCE	TARGETS	MINPODS
MAXPODS	REPLICAS		
backend-hpa	Deployment/backend	cpu: <unknown>/60%	1 4

TARGETS 当前显示 `<unknown>`，说明截图时 metrics-server 的 CPU 指标尚未返回可用数据，或指标采集尚未完成。HPA 资源已经创建，但课程要求中的压测扩容、停止压测后缩容截图还需要补充。建议等待 `kubectl top nodes` 和 `kubectl top pods` 有数据后执行：

```

kubectl top nodes
kubectl top pods
ab -n 10000 -c 200 http://1.92.103.90/api/ping
kubectl get pods -w
kubectl describe hpa backend-hpa

```

HPA 扩容不会在请求开始瞬间发生。metrics-server 存在采集周期，HPA 控制器也有评估间隔，因此负载升高到副本数增加之间会出现延迟。停止压测后，缩容同样会受稳定窗口和冷却策略影响，这可以避免短时波动造成副本频繁创建和删除。云平台按资源计费时，HPA 的价值在于低负载阶段减少副本占用，高负载阶段再补充计算能力，使任务可用性和资源成本之间取得平衡。

【截图占位：图 15 `kubectl top nodes` 或 `kubectl top pods`，证明 metrics-server 可用。】

【截图占位：图 16 `kubectl get hpa`，需显示 backend-hpa 的 min/max 和 CPU 目标。】

【截图占位：图 17 压测期间 `kubectl get pods -w`，需显示 backend Pod 数从 1 扩到 2 或更多。】

【截图占位：图 18 停止压测后 Pod 缩回的状态截图。】

三、第二部分：MPI 并行科学计算

3.1 环境部署

本项目选择课程任务书中的方向 B：MPI 并行科学计算。仓库中已包含 `mpi/mpijob.yaml`，该文件使用 Kubeflow MPI Operator 的 MPIJob 资源提交作业，配置为 `slotsPerWorker=2`、`Worker replicas=2`，总进程数可设置为 4。镜像地址当前为 `swr.cn-north-4.myhuaweicloud.com/cloud-compute-2026/mpi4py:latest`，需要在 SWR 中准备对应的 `mpi4py` 镜像后再提交作业。

该部分云端执行尚未完成。后续需要先部署课程提供的 MPI Operator：

```
kubectl apply -f mpi-operator.yaml
kubectl apply -f mpi/mpijob.yaml
kubectl get pods
kubectl logs <launcher-pod-name>
```

报告验收截图应包含 Launcher Pod 完成状态和日志中的 π 估算值。

【截图占位：图 19 `kubectl get pods` 显示 MPI Launcher/Worker Pod 状态。】

【截图占位：图 20 `kubectl logs <launcher-pod-name>`，需包含 π 估算结果。】

3.2 并行算法实现

算法题选择数值积分。`mpi/integral_serial.py` 是串行版本，使用中点公式对函数 $4/(1+x*x)$ 在 $[0,1]$ 区间积分，积分结果近似 π 。`mpi/integral_mpi.py` 是阻塞 MPI 版本，每个进程根据 rank 和 size 计算自己负责的区间，得到 `local_sum` 后通过

`comm.reduce(local_sum, op=MPI.SUM, root=0)` 汇总到 rank 0。rank 0 将总和乘以步长得到最终 π 估计值。

MPI 版本的区间划分方式为：总点数 `n` 按进程数 `size` 切分，每个进程计算 `[start_i, end_i)` 范围内的局部累加。末尾进程负责余数部分，避免 `n` 无法被进程数整除时丢失计算点。通信模式是典型的多对一 Reduce：所有 Worker 将局部和发送到 rank 0，rank 0 进行汇总和输出。

3.3 性能测试与 Amdahl 分析

性能测试表需要在 CCE 上实际运行后填写。任务书要求固定问题规模，如积分点数 10,000,000，在 1、2、4 个 MPI 进程下各运行 3 次取平均。建议记录如下表格：

进程数 p	第 1 次 /s	第 2 次 /s	第 3 次 /s	平均运行 时间 T(p) /s	实测加速 比 S	Amdahl 理 论值
1	待测	待测	待测	待填	1.00	1.00
2	待测	待测	待测	待填	待填	待填
4	待测	待测	待测	待填	待填	待填

实测加速比按 $S(p)=T(1)/T(p)$ 计算。Amdahl 理论值可用 $S=1/((1-f)+f/p)$ 表示，其中 `f` 是可并行比例。若用 4 进程实测加速比反推 `f`，可将公式变形为 $f=(1-1/S)/(1-1/p)$ 。实测值通常低于理论线性加速，原因包括 MPI 进程启动开销、Reduce 汇总同步开销、容器调度开销、节点间网络延迟和负载划分误差。

【截图占位：图 21 1、2、4 进程运行三次的命令输出。】

【截图占位：图 22 实测加速比与 Amdahl 理论加速比双折线图。】

3.4 非阻塞通信优化

`mpi/integral_mpi_nonblocking.py` 使用 `Irecv` 和 `Isend` 改写汇总通信。rank 0 先为每个非 0 进程创建接收缓冲区并调用 `Irecv`，Worker 使用 `Isend` 发送局部和，随后调用 `Wait` 确认发送完成。rank 0 使用 `MPI.Request.Waitall(requests)` 等待所有接收完成，再累加缓冲区中的局部结果。

非阻塞通信的优势来自计算和通信的潜在重叠。在本实验的积分任务中，每个进程主要耗时集中在本地循环计算，最终只发送一个 `double` 类型局部和，通信数据量很小。因此非阻塞版本的提升可能有限，甚至会因为额外请求管理开销而没有明显加速。若任务改为矩阵块交换、排序中的邻居交换或大规模边界数据传输，通信时间占比提高后，非阻塞通信更容易体现优势。

【截图占位：图 23 4 进程阻塞版与非阻塞版运行时间对比。】

四、总结与收获

本次课程设计把容器镜像、Kubernetes 编排、Redis 持久化、配置分离、LoadBalancer 暴露和 HPA 资源串联为一次完整的云平台实践。部署过程中，单个 YAML 字段或云厂商注解都可能影响最终结果。例如 SWR 镜像缺失会导致 Pod 进入 ImagePullBackOff，LoadBalancer Service 缺少华为云 ELB 自动创建注解会长期停留在 <pending>。这些问题不能只看 Pod 状态，需要继续检查 describe pod、describe svc 和事件信息，才能定位到镜像认证、镜像不存在或 ELB 创建条件不足等具体原因。

实验也体现了配置分离的实际意义。后端通过 ConfigMap 和 Secret 读取 Redis 地址与密码，避免把环境相关配置写死到代码中；前端通过 ConfigMap Volume 挂载 Nginx 配置，使反向代理规则从镜像中分离出来。Redis 使用 PVC 后，数据目录不再依附于单个 Pod 生命周期，为后续删除 Pod 后验证数据保留提供了基础。

MPI 部分展示了云原生环境中运行并行科学计算的基本方式。数值积分本身计算逻辑简单，适合观察串行、阻塞 MPI 和非阻塞 MPI 的差异。真正影响性能的因素不只包括算法的可并行比例，还包括进程启动、通信同步、容器调度和网络传输。后续完成 1、2、4 进程的实际测试后，可以结合 Amdahl 定律解释实测加速比与理论值之间的差距。

五、证据与截图清单

证据	文件
Ghostty 真实终端截图，含节点、Pod、Service、PVC、HPA、公网 API	report/evidence/ghostty-final-status-cropped.png
Ghostty 原始全屏截图	report/evidence/ghostty-final-status.png
最终部署文本记录	report/evidence/20260612-193650-final-success-status.txt
本地单元测试与公网 API 验证	report/evidence/20260612-194545-verification-complete.txt
初始部署状态记录，含镜像推送前的排 查状态	report/evidence/20260612-192127-initial-deploy-status.txt

附录：核心文件

类型	文件
后端代码	backend/app.py
后端依赖	backend/requirements.txt
后端 Dockerfile	backend/Dockerfile

类型	文件
前端页面	frontend/static/index.html
前端 Nginx 配置	frontend/nginx.conf
K8s 配置	k8s/00-config.yaml 至 k8s/05-hpa.yaml
MPI 串行程序	mpi/integral_serial.py
MPI 阻塞程序	mpi/integral_mpi.py
MPI 非阻塞程序	mpi/ integral_mpi_nonblocking.py
MPIJob	mpi/mpijob.yaml
代码仓库链接：待填。	